

The first subtask can simply be solved by traversing the monochrome neighbourhoods using a DFS or BFS. In that case, the complexity of one coloring operation is linear with respect to the number of pixels in a component (monochrome neighborhood). In the worst case, the component will have $O(R \cdot S)$ pixels, so the total time complexity of this approach equals $O(R \cdot S \cdot Q)$.

The second subtask is a special case in which the image is one dimensional. Some of the approaches are:

- Maintaining a set of components (continuous intervals) using an efficient data structure (such as `std::set` in C++) that enables fast retrieval of neighbouring elements and their merging (insertion and deletion).
- Maintaining a set of connected components using *union find*.
- Dividing an array into parts of length $O(\sqrt{S})$ and keeping a flag for each part which tells us whether the whole part is a single component (i.e. is it monochrome).

The third subtask is special in the following way: in each step either there will be no changes (bucket tool is filled with the same color as the pixel it is applied on) or the component of the pixel will change its color and connect itself with *all* of its neighboring components. Maintaining these components can be done using the *union find* data structure, along which we will keep track of a set of active neighbors (e.g. using `std::set` in C++). When traversing the adjacency list of the current component, i.e. when connecting it with one of its neighbours, we delete that neighbor from its set of active neighbors (and vice versa). There will be no more than $O(R \cdot S)$ connections and each has a complexity of deleting an element from the set, $O(\log(R \cdot S))$. The total time complexity of this approach is therefore $O(R \cdot S \cdot \log(R \cdot S))$.

On first glance, it may seem that the algorithm from the third subtask can be used to obtain the full score. But, the problem resides in the fact that we don't necessarily need to connect the current component to all of its neighbors in a single application of the bucket tool. Perhaps we can solve the problem by keeping the neighbours somehow grouped by color (e.g. using `std::map` in C++), but when changing color of a current component, we would still need to traverse over all of its neighbors in order to let them know we have changed our color. Let's keep all of these observations in mind, but return to the solution of the first subtask.

Note that, if the average size of components was sufficiently small, say 10 pixels, then the algorithm from the first subtask would be fast enough, i.e. $O(R \cdot S + Q \cdot 10)$. What if the average size of components was larger? Something like 100 pixels should be good enough as well, a 1000 should also be squeezable. What about 10000 pixels? Unlikely. Luckily, increasing the size of an average component decreases the number of components that have at least that many pixels. In other words, at each moment there are at most $R \cdot S/K$ components that have K or more pixels (we will choose the exact value of K later). This property could be of use. The intuition is as follows: we will use one of the naive approaches to solve for components with K or less pixels, and some other approach to solve for components with more than K pixels. The important thing is that this approach doesn't need to be efficient when the component has less than K pixels.

More precisely, for each component we track the following values: *color*, *size* and *a set of large neighbors* (those with more than K pixels – note that there is at most $R \cdot S/K$ of those). When connecting two

components, we use *union find* to maintain these values. Additionally, for large components we keep around a *list of its neighbours grouped by color*. We will explain the algorithm using the following three operations:

1. Operation $Color(x, y, color)$ – applies the bucket tool on a pixel (x, y) :
 - Component label c to which pixel (x, y) belongs to is determined.
 - We use the operation $NeighborsInColor(c, color)$ which determines the labels of neighboring components of the given color, $c'_1, c'_2, \dots, c'_i$.
 - These components (c'_i) are connected with the current component c ($c = Spoji(c, c'_i)$).
 - After connecting, we go through all big neighbors of a component and we notify them on the color change (i.e. the label of a current component c is added into lists of neighboring components for a given color).
2. Operation $NeighborsInColor(c, color)$ – finds neighboring components of a given color:
 - If component c is small, traverse all of its neighbors (at most $O(K)$ of them) and for each we check what color is it.
 - If component c is big, we use a list of neighbors grouped by color.
Observation: After traversing that list, it can (has to) be deleted (after connecting, there will no longer be a neighbor of a given color – they will be a part of the same component).
3. Operation $Connect(c_1, c_2)$ - connects components c_1 and c_2 :
 - Size of the resulting components equals to the sum of sizes of components c_1 and c_2 .
 - The list of big components is obtained using *small-to-large* technique. (It's possible that after this operation we have some labels in our list that have been destroyed in the meantime, i.e. became connected in some larger component. We leave this implementation detail to the reader to resolve). the implementation detail discussed further in the text).
 - In case of connecting two large components, we need to obtain a joint list of neighbors grouped by color. This is also done using *small-to-large* technique.

Operation $Connect$ will be done at most $O(R \cdot S)$ times. Smaller to larger technique secures the total amortized complexity of $O(R \cdot S \cdot \log(R \cdot S))$. The complexity of all operations $NeighborsInColor$ in case of small components is $O(R \cdot S \cdot K)$, while the amortized complexity in case of big components is $O(R \cdot S)$. Finally, the complexity of all $Color$ operations is $O(Q \cdot R \cdot S/K)$. The total complexity also depends on a parameter K and equals $O(R \cdot S \cdot \log(R \cdot S) + R \cdot S \cdot K + \frac{Q \cdot R \cdot S}{K})$. It follows that the optimal runtime should be obtained when $K = O(\sqrt{Q})$.